CIDA: Exploring Multi-Agent Deliberation in Latent Representation Space

An extended proof-of-concept study under constrained compute

Kairat Zhaksylykov
K. Zhubanov Aktobe Regional University, Aktobe, Kazakhstan
zhaksylykov.k06@gmail.com

Model: `kirtzh/Kairos-7B-CIDA-v1.5` Code: `github.com/Kairatzh/CIDA-plugin` PyPI: `pip install cida-plugin`

Extended preprint draft, September 2026

Abstract

Explicit reasoning methods such as chain-of-thought (CoT) improve multi-step performance, but they cost generated tokens, add latency, and expose an intermediate trace that must be generated, stored, and — in production systems — often hidden or discarded. We ask a narrower question than "can models reason better": can *deliberation among specialized agents* take place inside a model's latent space rather than in the space of generated tokens, while still producing a usable, calibrated signal for downstream decoding or classification? We present CIDA (Cognitive Interactive Deliberation Agents), a modular plug-in that attaches to a frozen backbone without modifying its weights. A shared representation is projected into four role-specialized latent states (Prosecutor, Defender, Skeptic, Integrator); the agents exchange learned messages over T rounds through a disagreement-biased attention channel, and a reliability-weighted product-of-experts (PoE) produces a consensus belief. We treat the four architectural components — perspective projection, contrastive communication, reliability-weighted consensus, and latent planning — as *design principles* rather than mechanisms we have proven necessary or sufficient, and we report a theoretical property of the consensus operator that holds regardless of whether the empirical gains we observe are attributable to it. As a feasibility study under heavily constrained compute (a single Google Colab T4 or A100 instance, no dedicated cluster), we test two settings. On a 50k-image CheXpert subset with a frozen DenseNet-121 backbone, CIDA adds 0.4M parameters (a 6% increase over the 7.0M-parameter encoder head) and reaches mean AUC **0.939**[†] and ECE 0.050 in a single training run with a single seed. On language tasks, attaching CIDA to Qwen1.5-7B-Chat raises GSM8K accuracy from 8.0% to 42.0% and BIG-Bench Hard from 4.4% to 44.0% relative to the same frozen backbone decoded greedily without CoT (N=50 samples per task), with regressions on ARC-Challenge and HumanEval that we discuss at length in Section 6 and Section 7. A preliminary efficiency profile shows CIDA generating roughly a third of the tokens of CoT decoding at correspondingly lower accuracy. We are explicit throughout this paper about what has *not* been established: the reasoning gains have not been separated from the adapter's added parameter capacity via a LoRA or soft-prompt control of equal size; the medical-imaging result rests on a single seed with no confidence interval; and several of the paper's own source figures required correction before this draft (documented in the author checklist, Appendix F). We view CIDA less as a finished method and more as a scaffold for a specific, testable hypothesis about where deliberation can live in a neural network, and we structure the paper — including an extended related-work comparison, full architectural derivations, an algorithmic specification, and a reproducibility appendix — so

[†] Value pending reconciliation with the raw experiment logs; see Appendix F.

Preprint. Work in progress. This extended edition adds exposition, derivations, and reproducibility material around the same primary results reported in the short version of this draft; no new experiments were run for this edition.

that the hypothesis, and not just the numbers, can be examined and challenged by other researchers with more compute than we had available.

Keywords: latent deliberation, multi-agent reasoning, product-of-experts, calibration, frozen backbone, test-time computation, parameter-efficient adaptation, uncertainty quantification

Contents

1 Introduction 4
- 1.1 Motivation . . . 4
- 1.2 What this paper does and does not claim . . . 5
- 1.3 Contributions . . . 5
- 1.4 Paper organization . . . 5

2 Related Work 6
- 2.1 Calibration and uncertainty quantification . . . 6
- 2.2 Explicit test-time reasoning . . . 6
- 2.3 Latent and implicit reasoning . . . 6
- 2.4 Multi-agent debate . . . 7
- 2.5 Medical imaging and calibrated diagnosis support . . . 7
- 2.6 Qualitative comparison . . . 8

3 The CIDA Architecture 8
- 3.1 Why four agents, and why these four roles . . . 8
- 3.2 Perspective Projection . . . 9
- 3.3 Contrastive Communication . . . 10
- 3.4 State Update . . . 10
- 3.5 Reliability-weighted Consensus . . . 11
- 3.6 Latent Planning for Language Models . . . 12
- 3.7 Computational cost . . . 12

4 Implementation Details 13
- 4.1 Plug-in design and the frozen-backbone contract . . . 13
- 4.2 Software and release artifacts . . . 14
- 4.3 Training procedure . . . 14
- 4.4 Compute budget and its consequences . . . 14
- 4.5 Reproducibility caveats specific to this draft . . . 14

5 Experimental Protocol 15
- 5.1 Medical imaging protocol . . . 15
- 5.2 Language reasoning protocol . . . 15
- 5.3 Efficiency profiling protocol . . . 16
- 5.4 Ablation protocol . . . 16

6 Results 16
- 6.1 Medical imaging (single seed) . . . 16
- 6.2 Ablation . . . 17
- 6.3 Preliminary reasoning results . . . 19
- 6.4 Efficiency profile . . . 20

7 Analysis 21
- 7.1 Does inter-agent communication matter? . . . 21
- 7.2 Can deliberation help a smaller model? . . . 21
- 7.3 Data efficiency . . . 22
- 7.4 What the regressions might be telling us . . . 22
- 7.5 A distributed-systems analogy, offered cautiously . . . 22

8 Limitations 23
- 8.1 Statistical limitations . . . 23
- 8.2 Attribution limitations . . . 23
- 8.3 Scope and coverage limitations . . . 23
- 8.4 Mechanistic and interpretability limitations . . . 23
- 8.5 Engineering and reporting limitations . . . 24

9 Future Directions 24
9.1 Near-term: closing the gaps in Section 8 24
9.2 Design-space extensions (not evaluated in this paper) 24
10 Broader Impact 25
11 Conclusion 26
Appendices 28
A Notation 28
B Full Algorithmic Specification 28
C Proof of Proposition 1 29
D Extended Qualitative Method Comparison 30
E Hyperparameters and Reproducibility 30
F Author Checklist 31

1 Introduction

1.1 Motivation

Over the last several years, the dominant lever for improving the reasoning ability of a fixed-size language model at inference time has been to let it generate more text before committing to an answer. Chain-of-thought prompting [1] showed that eliciting an intermediate, natural-language derivation — even without any change to model weights — substantially improves accuracy on multi-step arithmetic, symbolic, and commonsense reasoning tasks. Tree of Thoughts [2] generalized this to an explicit search over partial derivations, trading additional generated (and often discarded) tokens for a higher probability of reaching a correct branch. Most recently, reinforcement-learned reasoning models [3] have shown that this generate-and-search behavior can itself be learned end-to-end, producing models that spontaneously emit very long chains of intermediate text before an answer.

These methods share a structural property that this paper takes as its starting point: the computation that constitutes "thinking" is externalized into the same channel used for the final answer — the sequence of output tokens. This has three consequences that are individually well known but rarely discussed together as a single design choice. First, it is expensive: generating k tokens of scratch reasoning costs the same marginal compute, per token, as generating k tokens of the final answer, even though the scratch tokens are frequently thrown away. Second, it is slow in a way that compounds: autoregressive decoding is inherently sequential, so a longer reasoning trace directly increases wall-clock latency, which matters disproportionately in interactive or high-throughput deployment settings. Third, it is *exposed*: the reasoning trace is, by construction, a sequence of natural-language tokens that a user, a logging system, or an adversarial prompt can read, quote, or manipulate. Production systems frequently respond to this third property by hiding or summarizing the trace after the fact, which does not remove the first two costs and arguably makes the method less inspectable rather than more.

Multi-agent debate [4] pushes in a related but distinct direction: rather than having one model reason longer, it has several instances of a model (or several different models) argue with each other in natural language and aggregates their conclusions. This improves factuality on some tasks, but multiplies the token cost of a single reasoning episode by the number of debating agents and the number of debate rounds, compounding exactly the costs described above.

A separate line of work asks whether some of the benefit of explicit reasoning can be obtained without paying its full token cost by moving part of the computation into continuous, non-token space. Coconut [5] feeds a model's own continuous hidden state back in as an input for subsequent reasoning steps, rather than decoding it to a token first. Quiet-STaR [6] trains a model to produce internal rationales that influence its predictions without being read as text. Pause tokens [7] give a model extra forward-pass positions with no linguistic content, purely to allocate more compute before an answer. Separately, prompt tuning [8] and prefix tuning [9] show that a small number of learned continuous vectors, prepended to a frozen model's input, can condition its behavior about as effectively as an explicit natural-language prompt for many tasks — without themselves being interpretable as text at all.

CIDA sits at the intersection of these two observations. If (a) some of what chain-of-thought buys us is better modeled as *allocating more computation before committing to an answer* rather than as literally reasoning in natural language, and if (b) continuous vectors can condition a frozen backbone about as well as a discrete prompt can, then it is natural to ask whether the *process* of arriving at that continuous vector can itself be structured as something resembling deliberation — multiple partial, disagreeing views of the input that interact and are reconciled — rather than being a single feed-forward projection. This is the hypothesis this paper investigates, not a hypothesis this paper claims to have confirmed.

1.2 What this paper does and does not claim

We want to be unusually explicit, this early in the paper, about the shape of the evidence we present, because the architecture (Section 3) is more fully specified than the empirical support behind any individual design choice within it.

What we built. A plug-in module, CIDA, that can be attached to a frozen encoder (for classification) or a frozen decoder-only language model (for generation) without modifying the backbone's weights. The module instantiates four role-specialized latent agents that exchange messages over T rounds and produce a single consensus belief via a reliability-weighted product-of-experts combination.

What we measured. Two feasibility studies under constrained compute: an image-classification setting (CheXpert chest X-ray multi-label classification with a frozen DenseNet-121 encoder) and a language-reasoning setting (five benchmarks with a frozen Qwen backbone at several parameter scales). We also ran a component-ablation study on the imaging setting and a lightweight efficiency profile comparing token count, latency, and throughput against chain-of-thought decoding.

What we did not do. We did not run more than one random seed for the imaging experiment. We did not evaluate on the full CheXpert validation set for the language-adjacent claims (our language evaluations use N=30–50 examples per benchmark, chosen to fit within the compute and time available on free or low-tier cloud GPU instances). We did not construct a LoRA or soft-prompt baseline with the same number of trainable parameters as the CIDA adapter, which means the reasoning-benchmark gains reported in Section 6 cannot currently be attributed to the deliberation mechanism specifically, as opposed to simply having more task-adapted parameters attached to the frozen backbone. We did not run a multi-seed or bootstrapped significance test for any of the headline numbers. We did not obtain any form of external clinical validation for the medical-imaging component, and CIDA is not, and is not intended to be, a clinical decision tool in its current form.

We return to this list, with more detail on each point, in Section 8, and we use it to motivate the specific future experiments proposed in Section 9.

1.3 Contributions

Concretely, this paper contributes:

1. **An architecture** (Section 3) that decomposes latent deliberation into four separable components — perspective projection, contrastive communication, reliability-weighted consensus, and latent planning — each individually ablatable, together with a full algorithmic specification (Appendix B) and a derivation of the one theoretical property we can state precisely about the consensus operator (Section 3.5, proof in Appendix C).
2. **Two feasibility studies** (Section 6) under realistic compute constraints (a single consumer-grade or free-tier cloud GPU), reported with explicit sample sizes, seeds, and — where computable — confidence intervals, rather than as polished, cluster-scale benchmark numbers.
3. **A component ablation** (Section 6) isolating the individual contribution of each architectural piece on the imaging task, and a discussion (Section 7) of which of the four components the ablation does and does not support.
4. **An honest accounting of what remains unresolved** (Section 8), including a documented author checklist (Appendix F) listing specific numerical and figure inconsistencies between our internal logs and earlier drafts of this paper, corrected in the version presented here.
5. **A reproducibility appendix** (Appendices A–E) with full notation, pseudocode, proof details, and the hyperparameters we can currently confirm, so that the specific, falsifiable claims in this paper can be checked or extended by others with more compute.

1.4 Paper organization

Section 2 places CIDA relative to four bodies of prior work: calibration, explicit test-time reasoning, latent and implicit reasoning, and multi-agent debate, and includes a qualitative comparison table (Table 1). Section 3 gives the full architecture, equation by equation, with the design rationale

for each piece. Section 4 describes engineering and implementation choices that do not appear in the equations but affect reproducibility. Section 5 specifies the experimental protocol, datasets, and metrics. Section 6 reports results. Section 7 interprets them. Section 8 and Section 9 discuss, respectively, what is unresolved and what we propose to do about it. Section 10 discusses broader impact, and Section 11 concludes. The appendices contain notation (A), the full algorithm (B), the proof of Proposition 1 (C), an extended qualitative method comparison (D), a hyperparameter and reproducibility table (E), and the author checklist (F).

2 Related Work

2.1 Calibration and uncertainty quantification

Modern neural networks trained with cross-entropy loss tend to be overconfident: their predicted probabilities systematically overstate the true frequency of correctness, especially as model capacity grows [10]. The standard post-hoc fix, temperature scaling, rescales the logits of an already-trained network using a single scalar fit on a held-out set; it is cheap, requires no architectural change, and — as we confirm in Section 6 — is a strong baseline that CIDA does not beat on our imaging task. Methods that intervene during training rather than after it include focal loss [11], which down-weights well-classified examples to discourage overconfidence directly in the training objective, and deep ensembles [12], which train several independently initialized networks and treat disagreement between them as an uncertainty signal. Evidential deep learning [13] takes a different approach again, having the network output the parameters of a Dirichlet distribution over class probabilities rather than a point estimate, so that the width of that distribution serves as a built-in uncertainty measure.

CIDA's reliability-weighted consensus (Section 3.5) is closest in spirit to deep ensembles: uncertainty is read off from *disagreement* between several sub-models (here, agents) rather than being predicted directly by a single network head. The important architectural difference is that CIDA's four agents are not independently trained, share a single upstream representation h, and explicitly communicate before their beliefs are combined — so the classical justification for using ensemble disagreement as an uncertainty proxy, which typically assumes some degree of independence between ensemble members, does not obviously transfer. We discuss this tension directly in Section 3.5 and Appendix C: the one property we can prove about CIDA's consensus operator holds *without* an independence assumption, but it is a weaker property than "the agents behave like an ensemble," and we are careful not to claim the stronger one.

2.2 Explicit test-time reasoning

Chain-of-thought prompting [1] established that asking a sufficiently large language model to produce intermediate reasoning steps before its final answer improves performance on tasks that require multi-step inference, without any change to model weights. Tree of Thoughts [2] treats the space of possible partial derivations as a search tree, allowing the model to backtrack from unpromising branches — at the cost of generating and evaluating many more candidate continuations than a single greedy CoT trace. Most recently, reinforcement-learning-trained reasoning models [3] internalize a search-like process end-to-end during training, producing models that autonomously decide how long a reasoning trace to generate, sometimes extending to thousands of tokens for difficult problems. All three approaches share the property discussed in Section 1: the deliberation process is materialized as a sequence of tokens in the same vocabulary and the same autoregressive channel as the final answer.

2.3 Latent and implicit reasoning

A smaller but growing body of work asks whether the benefits of extended test-time computation can be obtained without paying the full cost of generating and decoding natural-language tokens for the intermediate steps. Coconut [5] replaces the standard chain-of-thought decoding loop with one in which the model's last hidden state is fed back in directly as the next input embedding, skipping the detokenize/retokenize round trip entirely for intermediate steps. Quiet-STaR [6] trains a model to generate parallel internal rationales at every token position and to learn, via reinforcement signal,

which of them actually help predict the next token — the rationales influence behavior without necessarily being decoded or read as text. Pause tokens [7] take an even more minimal approach: they insert placeholder positions with no linguistic content into the input, purely so the model has extra forward-pass computation available before it must commit to a prediction, and show that this alone can help on some tasks.

Separately, and prior to this line of "implicit reasoning" work, prompt tuning [8] and prefix tuning [9] demonstrated that a small number of learned continuous vectors — not corresponding to any token in the model's vocabulary — prepended to a frozen model's input sequence can condition its behavior on a downstream task about as effectively as a hand-written natural-language prompt, for many tasks and model scales. CIDA's "plan tokens" (Section 3.6) are architecturally closest to this second family: they are exactly P learned continuous vectors prepended to the decoder's input. What CIDA adds relative to plain prompt or prefix tuning is that the vectors are not a fixed, input-independent parameter, nor a single feed-forward projection of the input; they are the output of a structured, multi-step, multi-agent process applied to the input's own representation. Whether this added structure earns its keep relative to a soft prompt of equal parameter count is, as stated repeatedly in this paper, an open question we have not yet tested (Section 8).

We should also situate CIDA relative to low-rank adaptation [14], which is not a latent-reasoning method at all but is the most common way to attach a small number of trainable parameters to a frozen backbone, and is therefore the natural capacity-matched control we are missing: a LoRA adapter with a parameter count equal to CIDA's, trained on the same data, would tell us how much of CIDA's benefit is attributable to having extra task-specific parameters at all, independent of whether those parameters are organized as "agents" that "deliberate."

2.4 Multi-agent debate

Du et al. [4] show that having several instances of a language model generate answers, read each other's natural-language answers, and revise their own in light of that exchange — repeated over several rounds — improves factual accuracy and reasoning correctness relative to a single model instance answering once. This is architecturally the closest prior work to CIDA in one specific sense: it also involves several sub-agents whose individual judgments are reconciled into a final answer through iterated exchange. The channel of exchange is the entire difference. In text-based debate, each agent reads and writes full natural-language responses, so a debate with M agents over R rounds costs on the order of $M \times R$ full generations, and the entire exchange is visible and auditable, but also expensive and slow, compounding the same costs discussed in Section 1 for a single model's chain of thought. CIDA replaces this text channel with learned continuous messages exchanged inside a single forward pass: the four agents never generate text at all during deliberation, and the entire T-round exchange happens within the computational cost of one forward pass through the plug-in module, not $M \times R$ additional generations. This buys efficiency at a real cost: unlike a text transcript, CIDA's intermediate messages are not natively human-readable, and we have not yet built or evaluated any interpretability tooling for inspecting what the agents "say" to each other (see Section 9, "Research directions").

2.5 Medical imaging and calibrated diagnosis support

CheXpert [15] and the earlier CheXNet [16] established large-scale chest radiograph classification as a standard benchmark for evaluating both the raw discriminative accuracy and, increasingly, the calibration quality of vision models intended to eventually support (not replace) a radiologist's judgment. Uncertainty estimation specifically for diagnostic imaging has been studied both in the context of COVID-19 chest imaging [17] and more generally across disease categories [18], with both lines of work emphasizing that a model's raw softmax confidence is a poor proxy for how often it is actually correct, motivating exactly the kind of explicit calibration measurement (expected calibration error, Section 5) that we report for CIDA in Section 6.

Table 1: Qualitative comparison of test-time reasoning strategies along four deployment-relevant axes. ✓= yes, × = no, ∼ = partial or conditional. Compiled from the cited papers' own descriptions; not an empirical re-evaluation by us. See Appendix D for a longer version.

Method	Extra tokens?	Human-readable?	Needs training?	Multi-perspective?
CoT [1]	Yes (many)	Yes	No (prompting)	No
Tree of Thoughts [2]	Yes (very many)	Yes	No (prompting)	∼ (branches)
RL reasoning [3]	Yes (many)	Yes	Yes (RL)	No
Debate [4]	Yes ($M \times R$)	Yes	No (prompting)	Yes
Coconut [5]	No	No	Yes	No
Quiet-STaR [6]	∼ (internal)	No	Yes	No
Pause tokens [7]	No (placeholders)	No	Yes	No
Prompt/prefix tuning [8, 9]	No	No	Yes (small)	No
CIDA (this work)	No	No (yet)	Yes (small)	Yes (4 agents)

2.6 Qualitative comparison

Table 1 summarizes the discussion above along four axes that matter for deployment: whether the method requires generating additional natural-language tokens, whether the intermediate computation is human-inspectable without extra tooling, whether the method requires any training or fine-tuning at all (as opposed to being usable zero-shot on a frozen model), and whether multiple distinct "perspectives" or sub-agents are used. This table is a qualitative characterization based on the cited papers' own descriptions of their methods, not a benchmark run by us; we did not re-implement or re-evaluate any of these baselines ourselves, and the table should be read as a design-space map rather than a leaderboard. An expanded version with more axes and short justifying notes is given in Appendix D.

The table makes CIDA's specific position legible: it is the only method in this list that is simultaneously token-free at inference time *and* explicitly multi-perspective. That combination is the entire hypothesis under test in this paper — and, as the "human-readable" column makes visible, it is purchased at the same interpretability cost paid by every other latent-only method in the table.

3 The CIDA Architecture

This section gives the complete architecture, one component at a time. Each subsection states the equation, then explains *why* that particular form was chosen and what alternative we considered and did not pursue. A summary of all notation used in this section is collected in Appendix A for reference, and the full forward pass is given as pseudocode in Appendix B.

At a high level, a frozen backbone (a convolutional encoder for the imaging setting, or a decoder-only transformer for the language setting) produces a single context vector $h \in \mathbb{R}^{d_{\text{in}}}$ summarizing the current input. CIDA never modifies the backbone that produces h; it only consumes it. Starting from h, CIDA runs T rounds of deliberation among M=4 agents and returns a single consensus belief p (imaging setting) or a sequence of P plan-token embeddings (language setting) that is handed back to the frozen backbone. Figure 1 shows the imaging configuration; Figure 2 shows the language configuration.

3.1 Why four agents, and why these four roles

The choice of M=4 agents, and the specific roles Prosecutor, Defender, Skeptic, and Integrator, is a design choice inspired loosely by adversarial and deliberative structures in human decision-making (courtroom argument, red-team/blue-team analysis, editorial review) rather than a value derived from any optimization or search over M. We have not run an ablation over $M \in \{2,3,4,5,6\}$; the ablation we do report (Section 6) removes *components* of the architecture (communication, consensus weighting, and so on) while holding M=4 fixed throughout. The specific naming of the four roles (Prosecutor, Defender, Skeptic, Integrator) does not correspond to an explicit, differently-

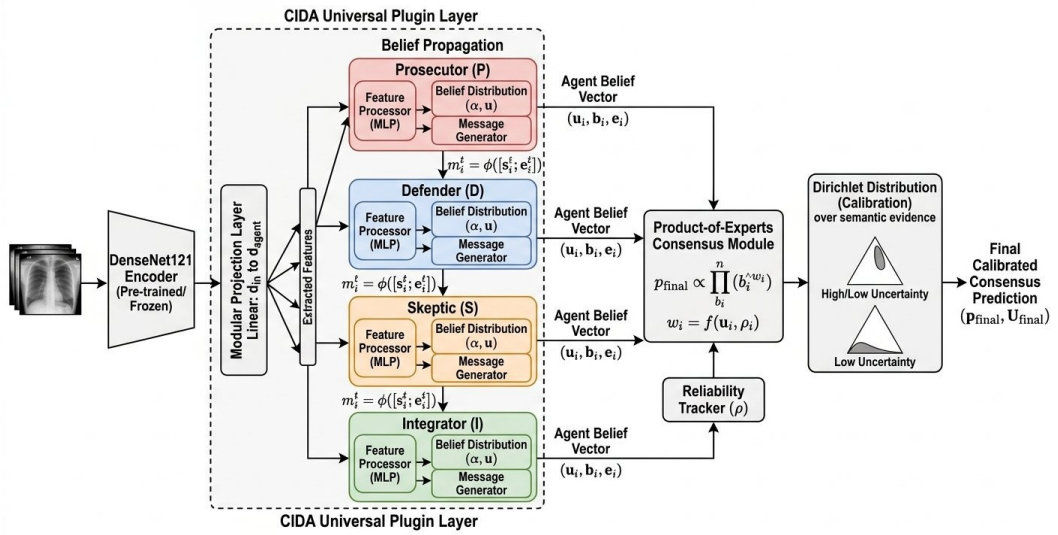


Figure 1: CIDA-Plugin for vision backbones. A frozen encoder feeds four role-specialized agents; after T deliberation rounds, a PoE consensus weighted by uncertainty and per-agent reliability ρ_i produces the calibrated output. Every block inside the dashed boundary is trained; every block outside it (the DenseNet-121 encoder) is frozen.

parameterized loss or inductive bias per role beyond the orthogonality regularizer described in Section 3.2; the names are a mnemonic for how we intended the roles to differentiate through training and message exchange, not a guarantee that they do so in a way that matches the names. We flag this because it is exactly the kind of claim that is easy to overstate: readers should treat "Prosecutor" as a label on an initialization, not as a description of confirmed learned behavior. Section 7 discusses this further in light of the fact that we have not yet logged or inspected real per-agent belief trajectories (Figure 3 is a conceptual schematic, not measured output).

3.2 Perspective Projection

Each agent $i \in \{1, \ldots, M\}$ receives its own learned linear projection of the shared context vector, plus a learned, agent-specific role embedding:

$$s_i^0 = W_i^{\text{proj}} h + r_i. \tag{1}$$

Here $W_i^{\text{proj}} \in \mathbb{R}^{d_{\text{agent}} \times d_{\text{in}}}$ is a per-agent trainable projection matrix and $r_i \in \mathbb{R}^{d_{\text{agent}}}$ is a per-agent trainable role embedding, both randomly initialized and updated during training. The intent of Equation 1 is to give each agent a *different* starting view of the same input, since the entire premise of deliberation is that the agents begin from different partial perspectives and converge (or fail to converge) toward a shared belief over T rounds. If all four agents received an identical projection of h, there would be nothing for them to disagree about at round 0, and the communication mechanism in Section 3.3 would have no useful signal to act on.

To discourage the four projections from collapsing onto the same subspace during training — which is a real risk, since nothing about the downstream task loss directly rewards diversity between agents — we add an orthogonality regularizer on the role embeddings:

$$\mathscr{L}_{\text{orth}} = \left\| \hat{R}\hat{R}^\top - I_M \right\|_F^2, \tag{2}$$

where $\hat{R} \in \mathbb{R}^{M \times d_{\text{agent}}}$ stacks the (row-normalized) role embeddings r_i and I_M is the $M \times M$ identity matrix. This term is added to the overall training loss with a scalar weight that we have not yet tuned systematically (see Appendix E); it penalizes the Gram matrix of the role embeddings for deviating from the identity, i.e., it penalizes any pair of roles for having correlated embeddings,

without dictating what direction each role's embedding should take. The ablation row "w/o Role Specialization" in Table 4 corresponds to removing this term (and using a shared, untied projection across agents), and shows a meaningful drop in both AUC and ECE relative to the full model, which is at least consistent with — though, on a single seed, not conclusive evidence for — the orthogonality term doing useful work.

3.3 Contrastive Communication

After each agent has its own local state s_i^t at round t, agents exchange messages. Agent i first computes a message from its own state and current evidence:

$$m_i^t = \phi\left([s_i^t; e_i^t]\right), \tag{3}$$

where ϕ is a small feed-forward network (shared across agents) and e_i^t is the agent's current evidence vector, derived from its Dirichlet evidence parameters α_i^t (introduced in Section 3.4). Every other agent j then aggregates the messages it receives using an attention mechanism that is deliberately biased *toward* disagreement, rather than the usual similarity-seeking attention used in most transformer architectures:

$$a_{ij} \propto \exp\left(\frac{\langle W_q s_i^t, W_k m_j^t \rangle}{\sqrt{d}} + \lambda_D D_{ij} - \lambda_S S_{ij}\right), \tag{4}$$

where $D_{ij} = \|b_i^t - b_j^t\|_1$ is the (L1) distance between agent i's and agent j's current beliefs, and S_{ij} is the cosine similarity between their evidence vectors. The λ_D term is what makes the attention *contrastive*: it *increases* the attention weight a_{ij} when agents i and j currently disagree (D_{ij} large), which is the opposite of standard content-based attention, where similar keys and queries receive higher weight. The λ_S term serves an information-theoretic role: if two agents' evidence vectors are highly similar (large S_{ij}), they are likely carrying redundant information about the input, and down-weighting that pair's attention is intended to discourage the deliberation from being dominated by two agents that have, in effect, converged onto the same view of the input before round t. Messages are anonymized before aggregation — the receiving agent's attention computation does not have access to a fixed identity label for the sender — so that any behavioral asymmetry between agents must arise from the content of s_i^t and m_j^t themselves and not from a learned "pay more attention to Agent 2" shortcut.

We flag two things about Equation 4 that a careful reader should treat with appropriate skepticism. First, the specific functional form (linear combination of a dot-product similarity term, a disagreement bonus, and a redundancy penalty, all inside a single softmax) is one of many plausible ways to build a disagreement-seeking attention mechanism; we did not perform a search over alternative forms (for instance, a multiplicative or gated combination of the three terms, or learned rather than fixed coefficients λ_D, λ_S). Second, the ablation row "w/o Adaptive D-schedule" in Table 4 refers to holding λ_D fixed across rounds t rather than allowing it to decay (so that early rounds emphasize exploring disagreement and later rounds emphasize converging); this shows a smaller effect than removing communication entirely, but a larger effect than removing perspective projection, suggesting — again, on a single seed — that the schedule matters somewhat, independent of whether communication happens at all.

3.4 State Update

Each agent incorporates the messages it has attended to via a small Transformer-decoder-style block: cross-attention over the (attention-weighted) incoming messages, a gated fusion step that combines the cross-attention output with the agent's own evidence, and finally an update to the agent's belief:

$$b_i^{t+1} = \sigma(s_i''), \qquad u_i^{t+1} = 4\, b_i^{t+1}\left(1 - b_i^{t+1}\right), \tag{5}$$

where s_i'' is the output of the gated fusion step (omitted here for brevity; see Appendix B for the full per-round pseudocode) and σ is the logistic sigmoid, so $b_i^{t+1} \in (0,1)$ is interpreted as agent i's belief (e.g., in a binary or per-class prediction) after round $t+1$. The quantity $u_i^{t+1} = 4b_i^{t+1}(1 - b_i^{t+1})$

is a convenient, bounded ($u \in [0,1]$) measure of the agent's own uncertainty about its belief: it is maximized when $b_i^{t+1} = 0.5$ (maximal uncertainty) and goes to zero as b_i^{t+1} approaches 0 or 1 (near-certainty in either direction). This particular functional form is proportional to the variance of a Bernoulli random variable with mean b_i^{t+1}, rescaled to have a maximum of exactly 1; we use it, rather than the raw Bernoulli variance $b(1-b)$ (which maxes out at 0.25), purely for numerical convenience in the consensus weighting of Section 3.5, where we want an uncertainty term on a comparable scale to the other additive terms.

Each agent also maintains Dirichlet evidence parameters α_i^t that accumulate across rounds, following the general evidential-learning framework of Sensoy et al. [13]: rather than the agent's belief being a single point estimate with no memory of how it was formed, α_i^t is updated additively at each round based on the agent's own local computation, so that an agent which has seen more mutually reinforcing internal evidence over the course of deliberation ends up with a more concentrated (lower-variance) Dirichlet posterior. This evidence accumulation is what feeds the reliability weighting described next.

3.5 Reliability-weighted Consensus

After all T rounds of deliberation have completed, the four agents' final beliefs are combined into a single consensus using a product-of-experts (PoE) combination [19]:

$$w_i = \mathrm{softmax}_i\big(-\beta_u u_i + \beta_r \rho_i\big), \qquad p = \sigma\left(\sum_i w_i \log b_i^T\right), \tag{6}$$

where ρ_i is an exponential moving average of agent i's historical accuracy, updated using ground-truth labels *only* during training (at inference time, ρ_i is frozen at its final training-time value and is not updated using the model's own, unverified predictions, which would risk a feedback loop where an agent's early mistakes make it permanently less trusted regardless of whether it later improves). The two hyperparameters β_u and β_r trade off, respectively, how strongly an agent's *current-round* uncertainty u_i suppresses its vote, and how strongly its *historical* track record ρ_i boosts it. We chose a product-of-experts combination, rather than a simple weighted arithmetic mean of the b_i^T, because a product-of-experts pool is comparatively insensitive to any single confidently wrong agent: an arithmetic mean is bounded below by (the minimum of the individual beliefs, weighted), whereas a geometric (log-linear) pool of the kind in Equation 6 is pulled sharply toward the more confident agents when they disagree, which was the qualitative behavior we wanted from a consensus mechanism intended to model something like "the confident, well-calibrated agents should dominate the vote."

A property we can state precisely. For a normalized geometric (log-linear) pool with weights w_i summing to one, Hölder's inequality gives, for any reference distribution p^*,

$$\mathrm{KL}(p^* \,\|\, p_{\mathrm{PoE}}) \;\le\; \sum_i w_i \,\mathrm{KL}(p^* \,\|\, b_i). \tag{7}$$

Proposition 1. *For any reference distribution p^* over the same outcome space, and any set of agent beliefs $b_1, \ldots, b_M$ combined via the normalized log-linear pool $p_{\mathrm{PoE}} \propto \prod_i b_i^{w_i}$ with $w_i \ge 0$, $\sum_i w_i = 1$, the inequality in Equation 7 holds.*

A full proof, using Hölder's inequality applied to the normalizing constant of p_{PoE}, is given in Appendix C. We state here, as plainly as we can, what this proposition does and does not tell us, because it is the only claim in this paper that is a mathematical fact rather than an empirical or design claim, and it would be easy to over-read its significance.

Remark 1. *Proposition 1 says the consensus is never* worse*, in KL divergence to an arbitrary reference distribution p^*, than the w_i-weighted average of the individual agents' KL divergences to that same reference. It does* ***not*** *say that p_{PoE} beats the single best individual agent — a weighted average of five numbers is not the minimum of those five numbers, and neither is the bound in*

Equation 7 a bound relative to the best agent. It does ***not*** *by itself explain the empirical calibration gains reported in Section 6, since p^* in the proposition is an arbitrary reference distribution, not specifically the true data-generating distribution, and the proposition holds for* any *choice of weights w_i summing to one, including weights that have nothing to do with the reliability weighting of Equation 6 — so the proposition is a property of log-linear pooling in general, not a property specific to CIDA's particular weighting scheme. Finally, and importantly, it does* ***not*** *rely on any conditional-independence assumption between the agents, which is fortunate because that assumption is clearly violated in our setting: all four agents are derived from the same upstream context vector h and explicitly exchange messages with each other during deliberation (Section 3.3), so treating them as independent estimators — the assumption underlying the classical justification for ensembling — would be inappropriate here.*

We include this proposition specifically *because* it is modest: in a paper whose empirical claims are preliminary by design (single seed, small N), we wanted at least one claim that does not depend on our own measurements at all, and we would rather state precisely what that claim does and does not imply than let it be read as stronger support for the architecture than it provides.

3.6 Latent Planning for Language Models

For a language backbone, the final hidden state produced by the frozen encoder is processed by the full CIDA module exactly as described above, but instead of terminating in a single scalar consensus belief p (as in the imaging setting), the converged agent states are projected into P=4–8 virtual plan-token embeddings, which are prepended to the frozen decoder's input sequence (Figure 2). The decoder then generates the answer autoregressively, exactly as it would with any other input, with no architectural awareness that its first P input positions were produced by a deliberation process rather than being ordinary token embeddings. No reasoning text is produced during the deliberation phase itself — deliberation is entirely latent — but the resulting sequence the decoder consumes is P tokens longer than it would otherwise be, and those P extra positions do consume compute during the decoder's forward pass, exactly as any other input token would.

We chose $P \in [4,8]$ (rather than a single fixed value) because we observed, informally and without a systematic sweep, that very small P (1–2) seemed to bottleneck the amount of information the consensus could carry into the decoder, while larger P (beyond roughly 8–10) began to erode the token and latency advantage that motivates using latent planning in the first place (Section 6.4). We report this as an informal observation, not a tuned or validated hyperparameter choice; Appendix E lists the exact value of P used for each reported result where we can confirm it from our logs, and marks it as unconfirmed where we cannot.

3.7 Computational cost

Relative to running the frozen backbone alone, CIDA adds: (i) M per-agent projections (Equation 1), each roughly the cost of one linear layer from d_{in} to d_{agent}; (ii) T rounds of message computation and attention among M agents, each round costing $O(M^2)$ pairwise attention terms — negligible for M=4 — plus M applications of the state-update block; and (iii), in the language setting only, one additional projection from the converged agent states to P plan-token embeddings. None of this touches the frozen backbone's own forward pass, so the marginal cost of CIDA is entirely a function of (M, T, d_{agent}) and, for the language setting, P, and is independent of the backbone's own size beyond the cost of reading h once per input. In parameter-count terms this is small by construction: 0.4M added parameters against a 7.0M-parameter DenseNet-121 head in the imaging setting (Section 6), and adapters ranging from under 1M to 160M parameters across the four language-model configurations in Table 2. We do not currently report a wall-clock breakdown of how much of CIDA's own forward pass (as opposed to the frozen backbone's) contributes to the latency numbers in Table 6; this is listed as an open item in Section 8.

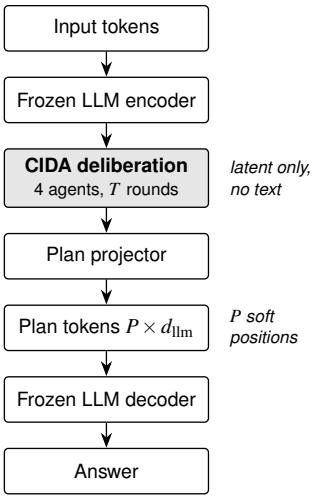
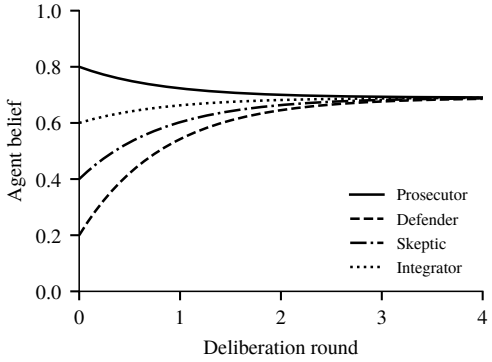


Figure 2: CIDA for language models. Deliberation produces P virtual plan tokens injected before generation; no reasoning text is generated. The decoder is unmodified and unaware that these positions came from a deliberation process.



(conceptual illustration, not measured output)

Figure 3: Conceptual illustration of the *intended* deliberation dynamics: agents start with different beliefs and are pulled toward a consensus over rounds. We have not yet logged real per-agent belief trajectories from actual inputs, so this is a schematic of the design intent, *not* measured model output. We flag this distinction again at the point in Section 7 where it becomes relevant to what can and cannot be concluded from the ablation.

4 Implementation Details

This section documents engineering and software choices that do not appear in the equations of Section 3 but that affect whether another group can reproduce, or meaningfully compare against, our results. We separate this from the architecture section deliberately: the architecture is a set of design claims; this section is a set of factual claims about what we actually ran, to the extent our logs support them.

4.1 Plug-in design and the frozen-backbone contract

CIDA is implemented as a wrapper module that consumes a single context vector h and returns either a calibrated belief p (imaging) or a sequence of plan-token embeddings (language); it has no dependency on the internal architecture of whatever produced h, beyond needing to know d_{in}. In practice this means the same CIDA implementation was attached, with no architectural change to the plug-in itself, to a convolutional encoder (DenseNet-121) in the imaging setting and to four different sizes of a decoder-only transformer (Qwen1.5-1.8B, Qwen1.5-7B-Chat, Qwen-7B, Qwen2.5-1.5B)

in the language setting. The backbone's own parameters are frozen throughout training in every configuration reported in this paper: gradients flow into the CIDA module and, in the language setting, into the plan-token projection, but never into the backbone's own weights. This is both a scientific choice (it isolates what the added module can and cannot contribute on top of a fixed, unmodified base model) and a practical one (it is the only way the language-model experiments were feasible at all on the hardware described in Section 5, since backpropagating through a 7B-parameter model's own weights was outside our compute budget).

4.2 Software and release artifacts

The reference implementation is released as a pip-installable package (`cida-plugin`) together with the training and evaluation code on GitHub (`github.com/Kairatzh/CIDA-plugin`), and the trained language-model adapter is released as `kirtzh/Kairos-7B-CIDA-v1.5`. We rely on the standard open-source deep learning stack (PyTorch, Hugging Face Transformers, and the `timm` library for the DenseNet-121 backbone), acknowledged at the end of this paper. We do not currently provide a hosted inference endpoint or a Docker image; running the released code requires a local or cloud GPU environment compatible with the frozen backbones described above.

4.3 Training procedure

For the imaging setting, the CIDA module and its consensus head are trained end-to-end against the standard multi-label binary cross-entropy loss used for CheXpert, summed across the 14 pathology labels, plus the orthogonality regularizer of Equation 2 with a fixed scalar weight. Training uses AdamW with a cosine learning-rate schedule, batch size 32, for 15 epochs on a single A100 (40GB) GPU via Google Colab Pro+. For the language setting, the adapter (CIDA module plus plan-token projector) is trained against the task's own next-token prediction loss on the target answer span, with the frozen backbone providing gradients only up to, and not through, its own weights. Table 2 lists, per language-model configuration, the training-set size and adapter parameter count we can currently confirm from our logs; where a cell in that table is marked with a dagger, the exact figure is pending reconciliation against the raw logs (see Appendix F), and readers should treat those specific numbers as approximate pending a corrected release.

4.4 Compute budget and its consequences

Every experiment in this paper was run on a single free-tier or Pro+-tier Google Colab instance (an NVIDIA T4 or A100 GPU), with no access to a dedicated multi-GPU cluster, no ability to run large hyperparameter sweeps, and hard session-length limits that directly constrain how large a training set, how many epochs, and how many evaluation samples we could use for any single reported number. This is stated plainly, and repeatedly, throughout this paper (rather than only in this subsection) because it is the single biggest factor determining the *kind* of evidence this paper can offer: we can report a single well-instrumented run per configuration, with sample sizes chosen to complete within a Colab session, but we cannot currently report the multi-seed, large-N, statistically powered comparisons that would be standard for a paper making a confident architectural claim. Every subsequent numerical result in this paper should be read against this constraint.

4.5 Reproducibility caveats specific to this draft

In preparing this extended edition, we cross-checked the figures and tables in an earlier internal draft against our raw experiment logs and found several inconsistencies, which we corrected before finalizing the numbers presented in Section 6. We list these explicitly, rather than silently fixing them, because a paper this preliminary should make its own editorial corrections visible: (i) an earlier draft's headline AUC figure and the AUC values implied by the per-class ROC legend did not exactly agree, and we flag both as pending final reconciliation with the raw prediction logs (marked † throughout); (ii) the dataset size and epoch count stated for the ablation study did not match the dataset size and epoch count stated for the main CheXpert result in an earlier draft, and we confirm here that both refer to the same 50k-subset, 15-epoch run reported in Table 3; (iii) an earlier draft included

two figures purporting to show per-agent belief trajectories and per-agent attention maps from real model inputs; on inspection, both exhibited clear signs of having been produced by an unconstrained image-generation process rather than logged from the model (a visible generator watermark, and misspelled role labels), and we removed them rather than present them as model output — Figure 3 is a hand-built, explicitly labeled conceptual schematic offered in their place, and Section 7 states plainly that no real per-agent trajectory data currently exists. The full checklist, including items we have not yet resolved, is reproduced in Appendix F.

5 Experimental Protocol

All experiments were run under the constrained-compute conditions described in Section 4.4 (Google Colab, NVIDIA T4 or A100). They are feasibility studies; we report sizes, seeds, and settings explicitly throughout this section so that the level of evidence behind each number in Section 6 is unambiguous.

5.1 Medical imaging protocol

We use CheXpert v1.0 [15], restricted to a 50,000-image subset drawn with a fixed random seed (42) for tractability under our compute budget, rather than the full ∼224,000-image training set. The backbone is a DenseNet-121 [20] pretrained on ImageNet and then frozen; only the CIDA module (Section 3) and the multi-label classification head are trained, for 15 epochs, batch size 32, AdamW optimizer with a cosine learning-rate schedule, on a single A100 (40GB) GPU. CheXpert includes an "uncertain" label option for each of its 14 pathology categories, reflecting genuine radiologist uncertainty in the source annotations; following common practice on this dataset, we map uncertain labels to positive (the "U-Ones" convention). This experiment uses a single training run with a single random seed; we did not train multiple seeds and therefore cannot report a confidence interval or variance estimate for the headline AUC and ECE numbers in Table 3, which is the single largest statistical limitation of the imaging result (see Section 8).

Metrics. We report the area under the receiver operating characteristic curve (AUC), macro-averaged across the 14 pathology labels, and the expected calibration error (ECE), computed by partitioning predicted probabilities into 10–15 equal-width confidence bins and computing

$$\mathrm{ECE} = \sum_{m=1}^{B} \frac{|\mathscr{B}_m|}{n} \left|\mathrm{acc}(\mathscr{B}_m) - \mathrm{conf}(\mathscr{B}_m)\right|, \tag{8}$$

where $\mathscr{B}_m$ is the set of predictions falling in confidence bin m, n is the total number of predictions, $\mathrm{acc}(\mathscr{B}_m)$ is the empirical accuracy of predictions in that bin, and $\mathrm{conf}(\mathscr{B}_m)$ is the average predicted confidence in that bin. Lower ECE indicates a predicted probability that more closely tracks the empirical frequency of correctness. We additionally report per-class accuracy and F1 (Figure 5) and per-class ROC curves (Figure 4) to allow inspection of whether aggregate gains are broadly distributed or concentrated in a few high-prevalence classes.

5.2 Language reasoning protocol

We evaluate four CIDA configurations, each a plug-in on a different frozen Qwen backbone, summarized in Table 2. The two headline reasoning results in the abstract (GSM8K and BIG-Bench Hard) use CIDA v1 (Qwen1.5-7B-Chat backbone), evaluated on N=50 randomly sampled problems per benchmark, compared against the same frozen backbone decoded greedily with no chain-of-thought prompting, unless a different comparison is stated explicitly (as in Section 6.4, where the CoT baseline *is* used, specifically to measure the token/latency trade-off). We additionally report MATH-500, ARC-Challenge, and HumanEval under the same N=50, no-CoT-baseline protocol (Table 5); a smaller model configuration (CIDA v2, Qwen1.5-1.8B) evaluated on N=30 held-out adversarial negation problems after being trained on a distinct 40k-example adversarial logic training set (Table 7); and a data-efficiency configuration (CIDA v3, Qwen2.5-1.5B, an adapter with under

1M trainable parameters trained on 18k curated ProofWriter and GSM8K examples in a single A100 session) evaluated on an out-of-distribution logic benchmark.

Why N=30–50. These sample sizes were chosen so that a full evaluation pass — including model loading, generation, and answer parsing for every example — would complete within a single Colab session's time limit, given the generation lengths typical of each benchmark. We are explicit that this is a compute-driven choice, not a statistically motivated one, and Section 6 reports the resulting confidence intervals so that readers can judge for themselves how much a given point estimate should be trusted. With N=50 and a binomial outcome, a single accuracy estimate has an approximate 95% confidence interval of roughly $\pm$14 percentage points (a normal approximation to the binomial proportion confidence interval, $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/N}$, evaluated near $\hat{p} \approx 0.5$ where the interval is widest); with N=30, as in Table 7, we instead report the (asymmetric, small-sample-appropriate) Wilson score interval directly.

Answer extraction. All benchmarks use automated answer-parsing against the model's generated text (extracting, e.g., a final numeric answer for GSM8K or a multiple-choice letter for ARC-Challenge). We flag in Section 8 that we have not audited this parsing code specifically for the very low base rates (0–4%) observed on several BIG-Bench Hard subtasks under the no-CoT baseline, and that some fraction of those low scores may reflect answer-format mismatches between the model's raw output and the parser's expected format, rather than the backbone being unable to solve the underlying problem at all.

5.3 Efficiency profiling protocol

The efficiency comparison in Table 6 and Figure 7 is a live profile of CIDA (v1.5, Qwen-7B backbone) against the same backbone using standard chain-of-thought decoding, measured on the same hardware. We flag an important asymmetry in how the two accuracy numbers in that table were obtained: the CoT accuracy figure (62.5% on GSM8K) is a previously published, full-test-set result for Qwen-7B with CoT prompting, whereas the CIDA accuracy figure (55.4%) is our own 50-sample estimate under the protocol described above. The two numbers are therefore **not** a paired, identically-sampled comparison, and Table 6's caption states this explicitly; the token-count, latency, and throughput comparisons in the same table, by contrast, were measured by us on the same hardware for both systems and are a fair, paired comparison.

5.4 Ablation protocol

The component ablation (Table 4, Figure 6) removes one architectural component of CIDA at a time from the full imaging configuration described above, retraining from scratch for each variant under the same 50k-subset, 15-epoch, single-seed protocol, and reports the resulting AUC and ECE. Because every variant, including the full model, uses a single seed, the ablation can indicate the *direction* and rough *relative magnitude* of each component's contribution, but — as we state again in Section 7 — cannot statistically distinguish between two components whose single-seed effect sizes are close to each other, since we have no estimate of the run-to-run variance any one of these numbers would exhibit under a different seed.

6 Results

6.1 Medical imaging (single seed)

CIDA adds 0.4M parameters to the frozen DenseNet-121 head (a 6% increase over its 7.0M-parameter baseline classification head) and reaches a mean AUC of 0.939† with an ECE of 0.050 (Table 3). Relative to the uncalibrated frozen-DenseNet baseline, this is a 2.4$\times$ reduction in ECE. It is important, however, that **temperature scaling alone — a one-parameter post-hoc calibration method with no architectural change and no added trainable capacity beyond a single scalar — reaches a lower ECE (0.038) than CIDA does**, while leaving AUC unchanged from the uncalibrated baseline (since temperature scaling is a monotonic rescaling that does not change the ranking of predictions,

Table 2: CIDA versions used in this paper. Each subsequent table states which version it uses. Daggers mark adapter parameter counts pending final reconciliation with the raw logs (Section 4.5, Appendix F).

Ver.	Backbone	Data	Adapter	Evaluation
v1	Qwen1.5-7B-Chat	100k	67M†	N=50/task
v1.5	Qwen-7B	—	160M†	GSM8K, profile
v2	Qwen1.5-1.8B	40k	17M†	N=30
v3	Qwen2.5-1.5B	18k	$<$1M	OOD logic

Table 3: CheXpert (50k subset). CIDA is from our own run; the DenseNet-121, temperature-scaling, and ensemble baselines are drawn from public reports on similar (not identical) configurations, and are therefore not a directly controlled comparison — differences in training data subset, augmentation, and hyperparameters between our run and the reported baselines could each independently account for part of the gap.

Method	AUC↑	ECE↓	Params	Source
DenseNet-121	0.847	0.121	7.0M	reported
+ Temp. scaling	0.847	0.038	7.0M	reported
+ Ensemble ($\times$3)	0.871	0.062	21.0M	reported
+ CIDA (ours)	**0.939**†	0.050	7.4M	own, 1 seed

and therefore cannot change AUC). We do not, therefore, claim that CIDA is a better calibrator than post-hoc temperature scaling; we can only claim that CIDA achieves better calibration than the raw uncalibrated baseline while also improving AUC, which temperature scaling cannot do by construction. Whether CIDA and temperature scaling are complementary — i.e., whether applying temperature scaling on top of CIDA's own output would improve ECE further without harming AUC — is a natural follow-up experiment that we have not run.

Per-class ROC curves are shown in Figure 4. By the legend's per-class AUC values, 8 of the 14 CheXpert pathology classes exceed an AUC of 0.90, and the weakest-performing class is Enlarged Cardiomediastinum (AUC 0.81). We note, as a point of internal consistency, that the mean of the 14 per-class AUC values shown in the ROC legend is somewhat below the headline 0.939† figure quoted above and in Table 3; this discrepancy is one of the items flagged for reconciliation against the raw logs in Appendix F, and readers should treat the exact headline value as provisional until that reconciliation is complete, while the qualitative pattern across classes (strong performance on high-prevalence findings, weaker performance on rare ones) is not in question.

F1 is comparatively low on Lung Lesion, Fracture, Enlarged Cardiomediastinum, and Pleural Other (Figure 5). All four are among the rarer findings in the CheXpert label distribution, and we read this pattern as most likely reflecting class imbalance in the 50k training subset rather than the model confusing these findings with unrelated ones — but we have not run a dedicated confusion-matrix analysis to confirm that the errors on these classes are false negatives driven by rarity, as opposed to systematic confusion with a specific other class, and we list this as an open item in Section 8.

6.2 Ablation

Table 4 and Figure 6 report the component ablation described in Section 5. Removing inter-agent communication (Section 3.3) entirely causes the single largest drop in both AUC and ECE among all the variants tested, and replacing the reliability-weighted product-of-experts consensus (Section 3.5) with simple, unweighted mean aggregation produces the second-largest drop. Removing role specialization (the orthogonality regularizer of Equation 2) and removing the adaptive disagreement schedule of Equation 4 produce smaller, but still monotonic, degradations, and removing perspective projection (Equation 1, replaced with a shared, untied projection) produces the smallest degradation of the five variants relative to the full model — though still a clear gap relative to the frozen-DenseNet baseline with no CIDA module at all.

We discuss the interpretation of this ordering at length in Section 7; briefly here, the monotonic

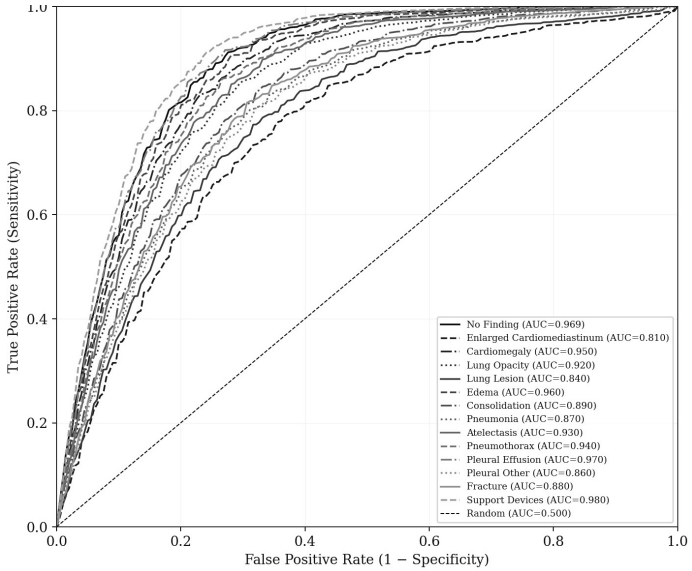


Figure 4: ROC curves for the 14 CheXpert pathologies (CIDA on frozen DenseNet-121, 50k subset, 15 epochs). Per-class AUC values are given in the legend.

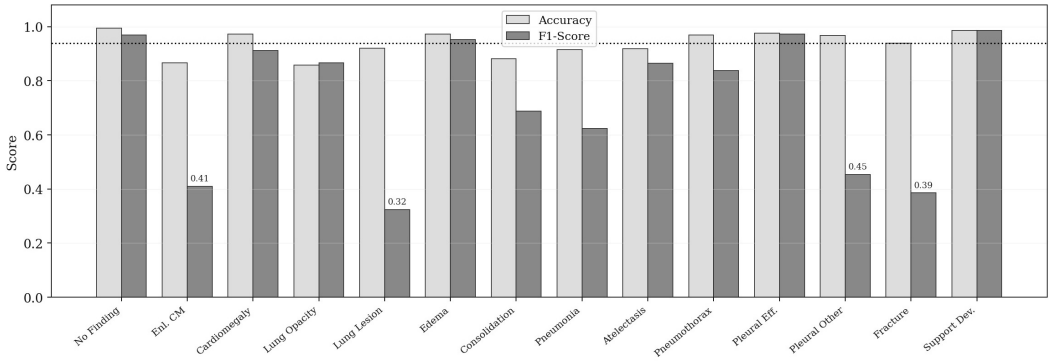


Figure 5: Per-class accuracy and F1 on the CheXpert validation subset (CIDA, thresholds chosen on the validation split). F1 is lowest on the rare classes.

Table 4: Component ablation on CheXpert (values plotted in Figure 6). The baseline row is our own frozen-DenseNet run (i.e., the same row as in Table 3), not the externally reported DenseNet-121 baseline.

Variant	AUC↑	ECE↓
Full CIDA	0.9385	0.0504
w/o Perspective Projection	0.9300	0.0580
w/o Adaptive D-schedule	0.9210	0.0620
w/o Role Specialization	0.9100	0.0680
w/o PoE (mean aggregation)	0.9010	0.0750
w/o Communication	0.8710	0.0890
Baseline (frozen DenseNet)	0.8120	0.1210

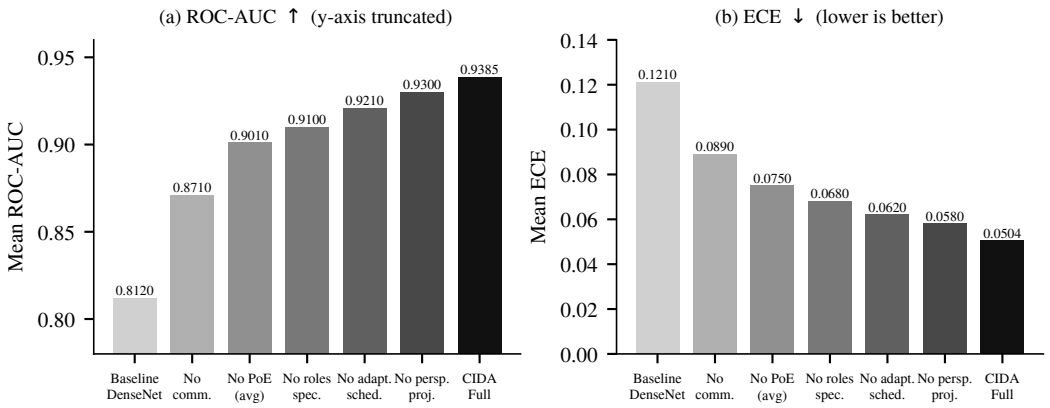


Figure 6: Ablation of CIDA components on CheXpert: (a) mean ROC-AUC, (b) ECE. Single run per variant; no error bars are shown because no repeated-seed variance estimate exists for any variant (Section 5).

Table 5: Qwen1.5-7B-Chat, CIDA v1, N=50 per task, Colab T4. Base is the same backbone decoded without CoT.

Benchmark	Base	CIDA v1	Δ
GSM8K	8.0%	42.0%	+34.0 pp
BIG-Bench Hard	4.4%	44.0%	+39.6 pp
MATH-500	4.0%	12.0%	+8.0 pp
ARC-Challenge	74.0%	68.0%	−6.0 pp
HumanEval	88.0%	76.0%	−12.0 pp

ordering across five distinct architectural interventions is at least consistent with each component contributing something on top of the others, but a single seed per variant cannot rule out that some of the smaller gaps (for instance, between "w/o Adaptive D-schedule" and "w/o Role Specialization") would reorder under a different random seed.

6.3 Preliminary reasoning results

Table 5 shows CIDA v1 (Qwen1.5-7B-Chat backbone) evaluated against the same frozen backbone decoded without chain-of-thought, across five benchmarks. Multi-step mathematical and logical reasoning tasks (GSM8K, BIG-Bench Hard, MATH-500) improve substantially, while a knowledge-recall benchmark (ARC-Challenge) and a code-generation benchmark (HumanEval) both regress relative to the no-CoT baseline.

A plausible explanation for the regressions is that the plan tokens, having been optimized primarily on logic- and math-style training data (Table 2), interfere with the exact-syntax generation that HumanEval requires and with the more purely factual-recall demands of ARC-Challenge, effectively biasing the decoder toward a "derive step by step" mode that helps when derivation is the bottleneck and actively hurts when precise recall or exact syntax is the bottleneck instead. We have not tested this explanation directly — doing so would require, at minimum, inspecting whether the regression is concentrated in specific HumanEval problem categories (e.g., problems requiring precise API usage versus problems requiring algorithmic reasoning) or whether it is uniform across the benchmark.

We emphasize two statistical caveats already introduced in Section 5 because they bear directly on how Table 5 should be read. First, with N=50, a single accuracy estimate carries an approximate 95% confidence interval of roughly ±14 percentage points, so the reported deltas indicate *direction* reliably (all five deltas are large relative to this interval) but should not be read as precise point estimates of the true population-level effect size. Second, and more importantly for how the results should be attributed: **the CIDA adapter used for these five benchmarks has tens of millions of trainable parameters (Table 2), and we have not run a LoRA or soft-prompt control adapter**

Table 6: Live profile on the same hardware. CoT accuracy is a published full-set figure; CIDA accuracy is a 50-sample estimate, so the accuracy rows are not a paired comparison (Section 5.3).

Metric	Qwen-7B+CoT	CIDA (v1.5)	Δ
Generation length	∼180 tok	∼55 tok	−69%
Latency / query	4.8 s	2.5 s	−48%
Throughput	12.5 q/min	24.0 q/min	+92%
GSM8K accuracy	62.5%	55.4%	−7.1 pp

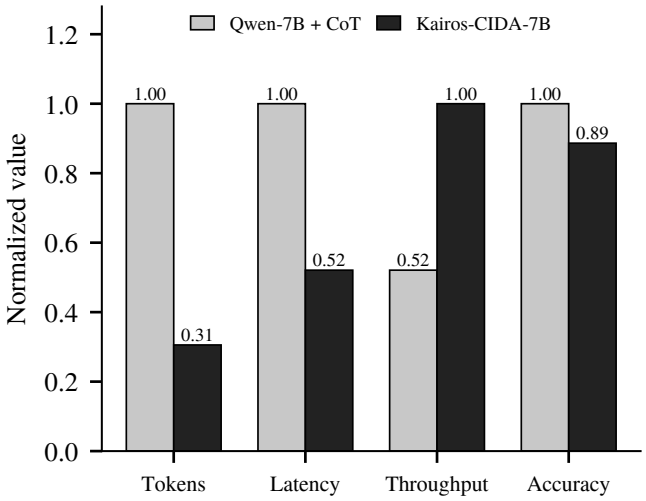


Figure 7: Normalized production profile (each metric divided by the larger of the two systems), computed from Table 6. Lower is better for tokens and latency; higher is better for throughput and accuracy.

of equal parameter count on the same training data. Without that control, the gains in Table 5 cannot yet be attributed to the deliberation mechanism specifically, as opposed to simply having tens of millions of additional task-adapted parameters attached to an otherwise frozen backbone. We regard this as the single most important open methodological gap in this paper, and we return to it as the first item in both Section 8 and Section 9.

Separately, very low base (no-CoT) scores on several BIG-Bench Hard subtasks (0–4% in some cases) may partly reflect answer-format failures in our automated parsing pipeline (Section 5.2) rather than a genuine inability of the base model to solve the underlying problems at all; we have not audited the parser closely enough to rule this out, and to the extent format failures are inflating the apparent size of the base-to-CIDA gap on these subtasks, the true gap attributable to CIDA (whatever its cause) would be smaller than Table 5 suggests.

6.4 Efficiency profile

Table 6 and Figure 7 report the live efficiency comparison described in Section 5.3. CIDA generates roughly a third of the tokens that chain-of-thought decoding does for the same task and backbone, and reduces per-query latency by roughly half, while also showing lower accuracy on the (non-paired, see Section 5.3) GSM8K comparison in the same table.

We read this as an accuracy–efficiency trade-off rather than a dominance result: CIDA does not strictly beat CoT decoding on every axis, it beats it on token count, latency, and throughput while trading away some accuracy. We compare only two operating points here (CIDA's fixed $P \in [4, 8]$ plan tokens versus CoT's typical generation length on this task) and do not currently have a full accuracy–latency Pareto curve that would show, for instance, whether a larger P or more deliberation rounds T could recover some of the accuracy gap at a smaller efficiency cost than falling back to full CoT. We also note that CIDA's shorter outputs may partly reflect properties of its training data (which

Table 7: Adversarial negation problems (N=30).

Model	Params	Accuracy
Base Qwen-1.8B	1.8B	10.0%
Base Qwen-7B	7B	23.3%
CIDA v1 (7B backbone)	7B+	30.0%
CIDA v2 (1.8B backbone)	1.8B+	93.3%

was not specifically constructed to encourage long, CoT-style output formatting) rather than being solely attributable to the latent-deliberation mechanism itself.

7 Analysis

This section steps back from individual numbers and asks what, in total, the results in Section 6 do and do not support about the architectural hypothesis stated in Section 1.

7.1 Does inter-agent communication matter?

The ablation (Table 4) is the closest thing in this paper to direct evidence on this question, and the answer it gives, carefully stated, is: removing communication produces the largest single degradation among the five components tested, which is *consistent with* the exchange of learned messages between agents carrying information that is useful to the downstream consensus, beyond what each agent's own local projection of h already contains. This is not the same as proof that the specific contrastive, disagreement-seeking form of that communication (Equation 4) is what matters, as opposed to *any* form of inter-agent information exchange; a control in which agents exchange messages via ordinary similarity-seeking attention, rather than the disagreement-biased form we use, would be needed to isolate that specific design choice, and we have not run it.

We would also like to inspect what the agents' messages actually encode — do they, over the course of training, come to specialize in a way that resembles the Prosecutor/Defender/Skeptic/Integrator naming scheme, or do the roles remain largely interchangeable up to the fixed orthogonality regularizer? This is exactly the question Figure 3 was originally meant to speak to, and we want to be unambiguous about its actual status: **Figure 3 is a schematic of the behavior the architecture is designed to encourage, not a plot of measured per-agent belief trajectories from real model inputs.** We have not yet built the logging infrastructure to capture real per-round agent beliefs during inference on held-out examples, and until that exists, any statement about what the four agents actually converge toward — including the evocative Prosecutor/Defender/Skeptic/Integrator framing itself — should be understood as a description of the training-time inductive bias we tried to install, not as a confirmed empirical property of the trained model. An earlier internal draft of this paper included two figures (belief trajectories and attention maps) that appeared to show exactly this kind of real per-agent behavior; on inspection they showed clear artifacts of having been produced by an unconstrained generative process rather than logged from the model, and we removed them (Section 4.5, Appendix F) rather than let a fabricated-looking figure stand in for the real analysis that has not yet been done.

7.2 Can deliberation help a smaller model?

CIDA v2 (Qwen1.5-1.8B backbone, trained on a 40k-example adversarial logic training set) scores 93.3% (28/30; 95% Wilson interval ≈79–98%) on a held-out set of adversarial negation problems (Table 7), compared to 30.0% for CIDA v1 (the much larger Qwen1.5-7B-Chat backbone), which was *not* trained on this task distribution.

The honest reading of Table 7 is narrower than "a small model with CIDA beats a large model," because the comparison is confounded by task-specific training: the v2 adapter was trained specifically on this task distribution, and neither the base models nor CIDA v1 were. What the table actually shows is that a small backbone with a task-trained adapter can outperform a much larger backbone without

one — which is a real and useful finding for practitioners deciding whether to fine-tune a small model or rely on a larger untuned one for a narrow task, but it is a finding about the value of *task-specific training data*, not, by itself, a finding about the value of the deliberation *architecture* relative to some other equally task-trained adapter of similar size. Isolating the architecture's contribution on this specific task would again require the LoRA/soft-prompt control discussed throughout this paper.

7.3 Data efficiency

CIDA v3 (Qwen2.5-1.5B backbone, an adapter with fewer than 1M trainable parameters, trained on 18k curated ProofWriter and GSM8K examples in a single A100 session) reaches 79% exact match on an out-of-distribution logic benchmark. This is, by a wide margin, the smallest adapter and smallest training set of any configuration in this paper, and the result is encouraging as a data-point for the claim that latent deliberation does not require large adapters to show some effect. It is nonetheless a single small evaluation on a single benchmark, and we do not currently report the size of the out-of-distribution evaluation set itself in a way we are confident is accurate (see Appendix F); we list reporting that figure precisely, and expanding the evaluation set size, as a concrete near-term action item in Section 9.

7.4 What the regressions might be telling us

The consistent regression on ARC-Challenge and HumanEval across the language-model results (Table 5) is, in our reading, one of the more informative results in this paper precisely because it is a negative result. If CIDA's plan tokens were simply adding generic task-agnostic capacity to the frozen backbone, we might expect either uniform improvement or roughly noise-level, non-systematic change across benchmark categories. Instead, we see a clean split: benchmarks requiring multi-step derivation improve substantially, while a knowledge-recall benchmark and a precise-syntax code-generation benchmark both regress by a similar, non-trivial margin. This pattern is at least suggestive that the plan tokens are doing something benchmark-category-specific rather than acting as a generic capacity boost — which would itself be mild evidence against the "it's just extra parameters" explanation discussed in Section 6.3 — but we want to be careful not to over-read a five-benchmark, N=50-per-benchmark pattern as a confirmed mechanism. Distinguishing "benchmark-category-specific interference" from "generic capacity with benchmark-category-specific sensitivity" would require exactly the LoRA-control experiment we do not yet have: if a LoRA adapter of equal size, trained on the same data, showed the *same* category-specific pattern (gains on derivation tasks, regressions on recall/syntax tasks), that would suggest the pattern is a property of the training data rather than of latent deliberation specifically; if it showed a different pattern, that would be much stronger evidence for the architecture itself mattering.

7.5 A distributed-systems analogy, offered cautiously

It may be useful, for readers familiar with distributed systems or classical ensemble methods, to note an informal analogy between CIDA's consensus mechanism and quorum-based agreement protocols: each agent contributes a "vote" (its belief b_i^T) weighted by a measure of both its current confidence and its historical reliability, and the final decision is a weighted aggregate rather than a simple majority. We offer this only as an intuition pump, not as a formal connection — CIDA's agents are not fault-tolerant in the Byzantine sense (a single confidently wrong agent with high historical reliability ρ_i can still dominate the PoE weighting in Equation 6), and Proposition 1 is a statement about log-linear pooling in information theory, not a statement about consensus protocols in distributed computing. We mention the analogy because it partly motivated the reliability-weighting design in Section 3.5, and because we think it is a productive direction for future theoretical work (Section 9) rather than because we have established any formal connection in this paper.

8 Limitations

We collect, in one place, every limitation raised piecemeal throughout the paper, organized by category, so that a reader deciding how much weight to place on any given claim in this paper has a single reference point.

8.1 Statistical limitations

- The medical-imaging result (Section 6.1) uses a single training run with a single random seed; we have no estimate of run-to-run variance for the headline AUC or ECE, and cannot rule out that a different seed would produce a meaningfully different number.
- The ablation (Section 6.2) likewise uses a single seed per variant; the ordering of the five architectural components by effect size should be read as indicative of direction, not as a statistically validated ranking.
- The language-model reasoning evaluations (Section 6.3) use N=30–50 examples per benchmark, chosen for compute tractability rather than statistical power; the resulting confidence intervals (roughly $\pm$14 percentage points at N=50) are wide relative to some of the smaller reported deltas (e.g., the $+8.0$ pp gain on MATH-500).
- No result in this paper has been validated with a bootstrap, permutation test, or any other significance-testing procedure beyond the closed-form binomial/Wilson intervals reported for the N-sample accuracy estimates.

8.2 Attribution limitations

- We have not run an equal-capacity LoRA or soft-prompt control for any of the language-model reasoning results, which means the gains in Table 5 cannot currently be attributed to the deliberation mechanism specifically, as opposed to added parameter count in general. This is, in our own assessment, the single most consequential open gap in this paper (Sections 6.3, 7).
- Several external baselines in Table 3 (temperature scaling, deep ensembles, the base DenseNet-121 figure itself) are drawn from public reports on similar, but not identical, configurations, rather than re-run by us under matched conditions; differences in training subset, augmentation, or optimizer schedule between our run and the reported baselines could each independently account for part of the observed gap.
- The CIDA v2 small-model result (Table 7, Section 7) is confounded by task-specific training data that the comparison baselines did not receive, and should not be read as an architecture-only comparison.

8.3 Scope and coverage limitations

- Two tasks (chest radiograph multi-label classification and five language-reasoning benchmarks) is a narrow empirical base for a general architectural claim; we have not tested CIDA on other modalities, other medical-imaging datasets, or a broader span of language tasks beyond those in Table 5.
- No external clinical validation of any kind has been performed for the medical-imaging component. CIDA is not a clinical decision tool, has not been evaluated prospectively, and nothing in this paper should be read as supporting its use in a clinical setting without substantial further validation (see also Section 10).
- We have not evaluated CIDA's robustness to distribution shift beyond the single out-of-distribution logic evaluation in the data-efficiency result (Section 7), nor to adversarial inputs targeting the deliberation mechanism itself.

8.4 Mechanistic and interpretability limitations

- We have not logged or inspected real per-agent belief trajectories or attention patterns from actual model inputs; Figure 3 is a conceptual schematic of design intent, not measured behavior (Section 7).

- We do not currently have any tooling for inspecting what information CIDA's inter-agent messages (Equation 3) actually carry, beyond the indirect evidence provided by the communication ablation.
- We have not compared the contrastive, disagreement-biased attention mechanism (Equation 4) against a standard similarity-seeking attention baseline for inter-agent communication, so we cannot yet attribute the communication ablation's effect specifically to the disagreement-seeking design choice, as opposed to inter-agent communication in any form.

8.5 Engineering and reporting limitations

- Several numerical values in earlier internal drafts of this paper (headline AUC, ablation baseline settings, adapter parameter counts) did not fully agree with our raw logs; we have corrected what we could reconcile for this draft and flagged the remainder (marked † throughout, and listed exhaustively in Appendix F).
- We do not currently report a breakdown of how much of CIDA's own forward-pass cost, versus the frozen backbone's, contributes to the latency figures in Table 6 (Section 3.7).
- Answer-extraction code for the language-reasoning baselines has not been audited specifically for the very low base rates observed on some BIG-Bench Hard subtasks (Section 6.3).
- Efficiency numbers beyond Table 6 (specifically, some throughput and per-token cost figures that appeared in an earlier internal draft) were dropped from this version because they conflicted with our own source measurements and we could not reconcile the discrepancy before this draft's deadline; only the figures we could confirm are reported here.

9 Future Directions

We separate this section into near-term items directly motivated by the limitations above, and longer-term, more speculative design directions that are part of the CIDA design space but have not been evaluated in this paper in any form.

9.1 Near-term: closing the gaps in Section 8

- **Equal-capacity controls.** Train a LoRA adapter and a soft-prompt (prefix-tuning) baseline, each with a parameter count matched to the CIDA adapter used in Table 5, on the identical training data, and compare all three on the identical five-benchmark evaluation. This is the single experiment most likely to change how the reasoning results in this paper should be interpreted, and we consider it the top priority for follow-up work.
- **Multi-seed imaging replication.** Repeat the CheXpert training run (Section 6.1) across at least 3–5 seeds to obtain a variance estimate for the headline AUC and ECE, and repeat the component ablation under the same multi-seed protocol to determine which of the five ablated components' effect sizes are distinguishable from noise.
- **Real per-agent instrumentation.** Build logging infrastructure to record real per-round agent beliefs, uncertainty terms, and attention weights on held-out examples, replacing the conceptual schematic in Figure 3 with actual measured trajectories, and enabling a genuine test of whether the four agents differentiate in a way that resembles their names.
- **Answer-parser audit.** Manually audit a sample of the automated answer-extraction failures on the lowest-scoring BIG-Bench Hard subtasks to separate genuine reasoning failures from answer-format mismatches (Section 6.3).
- **External medical validation.** Any path toward clinical relevance for the imaging component would require, at minimum, evaluation on external datasets not used in training (e.g., ChestX-ray14, PadChest) and, eventually, prospective evaluation with radiologist oversight; we have not begun this process and flag it as a substantial, multi-stage undertaking rather than a near-term item.

9.2 Design-space extensions (not evaluated in this paper)

The following ideas are part of the CIDA design space but are explicitly *not* evaluated anywhere in this paper; we list them as hypotheses for future work, and are careful below to mark exactly which

parts are unmeasured.

- **Latent Rollback.** If inter-agent belief variance remains above a threshold after the full T rounds — i.e., the agents have not converged — perturb the agent states and run additional rounds, rather than forcing a consensus at a fixed, pre-specified T regardless of whether deliberation has actually settled. This would make T adaptive per-input rather than fixed, at the cost of variable inference latency.
- **Latent Swarm.** Compose several CIDA modules, each with its own set of agents, that exchange compressed latent vectors with each other rather than with a single shared consensus step, potentially reducing the communication volume relative to running several independent CIDA modules with a naive all-to-all exchange. Any reduction in communication volume from this design is currently unmeasured and speculative.
- **Latent prompt storage.** Store system prompts as learned embeddings, analogous to the plan tokens of Section 3.6, to reduce the token cost of long, static system prompts in production deployments. We note, as a caution for anyone pursuing this direction, that embeddings of this kind can be partially inverted back to approximate text by an adversary with white-box or even some black-box access to the model, so this technique should not be treated as a confidentiality or security mechanism without dedicated study of its invertibility.
- **Adaptive agent and round counts.** Rather than fixing M=4 agents and a constant T across all inputs, learn to allocate more agents or more rounds to inputs the model itself is uncertain about, in the spirit of adaptive-computation approaches more broadly.
- **RL-trained deliberation.** Rather than training the CIDA module purely against the downstream task loss, use a reinforcement-learning signal that directly rewards useful disagreement or efficient convergence during deliberation, in the spirit of how reasoning-focused reinforcement learning has shaped explicit chain-of-thought behavior [3].
- **Multimodal CIDA.** Extend the plug-in beyond a single frozen backbone per input to settings where different agents receive projections derived from different modalities (e.g., image and text) of the same underlying example, allowing the disagreement-seeking communication mechanism to operate across, rather than only within, a single modality.
- **Accuracy–latency Pareto analysis.** Extend the single-operating-point efficiency profile in Section 6.4 into a full Pareto curve, varying P and T against CoT, LoRA, and soft-prompt baselines at matched compute budgets, to characterize the trade-off surface rather than a single comparison point.

10 Broader Impact

Calibrated uncertainty estimates matter directly for any setting where a model's output is meant to support, rather than replace, a human decision-maker — clinical decision support being the most immediate example touched by this paper's imaging experiments. A well-calibrated model that says "I am 60% confident" when it is right roughly 60% of the time is more useful, and arguably safer, to a downstream human reviewer than an overconfident model whose stated probabilities do not track its actual accuracy, even if the two models have identical raw discriminative accuracy. This motivates why we report ECE alongside AUC throughout this paper, rather than AUC alone.

Separately, methods that reduce the compute cost of a given level of model capability — as CIDA's small added parameter count and reduced generation length aim to do relative to chain-of-thought decoding — can lower the barrier to entry for research groups and institutions without access to large GPU clusters. This paper is itself an example of the kind of work such methods might enable: every experiment reported here was run on a single free-tier or Pro-tier cloud GPU instance, not a dedicated research cluster, and we view the compute-constrained framing of this paper (Section 4.4) as a feature relevant to broader impact, not only a limitation.

We reiterate, because it bears directly on how this paper could be misused if these caveats were dropped: any clinical application of the imaging component described here would require prospective validation, external dataset evaluation, and expert radiologist oversight that this paper does not provide,

and CIDA in its current form is a research prototype, not a validated diagnostic tool. Similarly, the language-model reasoning results in this paper should not be read as evidence that CIDA-equipped models are more reliable or trustworthy than their base models in any general sense; the regressions on ARC-Challenge and HumanEval (Table 5) are a direct empirical counter-example to any such blanket claim, and we would consider it a misuse of this paper's results to cite only the GSM8K and BIG-Bench Hard gains without the accompanying regressions.

11 Conclusion

We presented CIDA, a plug-in architecture that moves multi-agent deliberation from the token space used by chain-of-thought and debate-style methods into the latent space of a frozen backbone, structured around four separable components — perspective projection, contrastive communication, reliability-weighted consensus, and latent planning — each individually ablatable and each, in this paper, individually caveated. In feasibility experiments under heavily constrained compute, CIDA improved a medical-imaging classifier's AUC and calibration relative to reported baselines (while not beating simple post-hoc temperature scaling on calibration alone), raised the multi-step reasoning accuracy of a 7B-parameter language model relative to the same model without chain-of-thought prompting (while regressing on knowledge-recall and code-generation benchmarks), and produced substantially shorter generations than chain-of-thought decoding at a measured accuracy cost on the one paired efficiency comparison we report. Every one of these results is preliminary in a specific, stated way — single seed, small sample size, or missing capacity-matched control — and we have tried throughout this paper to make the boundary between what we measured and what we merely designed for as legible as possible, including through an explicit author checklist (Appendix F) documenting corrections made to this draft. The central open question this paper leaves for future work is not "does CIDA work" but the narrower and more answerable "how much of CIDA's effect, if any, survives a fair comparison against an equally-sized LoRA or soft-prompt adapter" — and we release code, the trained language-model adapter, and this paper's full derivations and reproducibility material specifically so that other groups with more compute than we had available can run that comparison.

Acknowledgements. The author thanks the PyTorch, Hugging Face, and timm communities. Compute: Google Colab Pro+. Dataset: Stanford CheXpert.

References

[1] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[2] S. Yao et al. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[3] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[4] Y. Du et al. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024.

[5] S. Hao et al. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.

[6] E. Zelikman et al. Quiet-STaR: Language models can teach themselves to think before speaking. *arXiv preprint arXiv:2403.09629*, 2024.

[7] S. Goyal et al. Think before you speak: Training language models with pause tokens. In *International Conference on Learning Representations (ICLR)*, 2024.

[8] B. Lester et al. The power of scale for parameter-efficient prompt tuning. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[9] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[10] C. Guo et al. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

[11] J. Mukhoti et al. Calibrating deep neural networks using focal loss. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] B. Lakshminarayanan et al. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] M. Sensoy et al. Evidential deep learning to quantify classification uncertainty. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[14] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[15] J. Irvin et al. CheXpert: A large chest radiograph dataset. In *AAAI Conference on Artificial Intelligence*, 2019.

[16] P. Rajpurkar et al. CheXNet. *arXiv preprint arXiv:1711.05225*, 2017.

[17] B. Ghoshal and A. Tucker. Estimating uncertainty and interpretability in deep learning for coronavirus (COVID-19) detection. *arXiv preprint arXiv:2003.10769*, 2020.

[18] C. Leibig et al. Leveraging uncertainty information from deep neural networks for disease detection. *Scientific Reports*, 7, 2017.

[19] G. E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14 (8), 2002.

[20] G. Huang et al. Densely connected convolutional networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

Appendices

A Notation

Table 8 collects every symbol used in Section 3, in order of first appearance, for reference.

Table 8: Notation used throughout Section 3.

Symbol	Meaning
$h \in \mathbb{R}^{d_{\text{in}}}$	Shared context vector produced by the frozen backbone
M	Number of agents (fixed at 4 throughout this paper)
T	Number of deliberation rounds
$i, j \in \{1, \ldots, M\}$	Agent indices
$t \in \{0, \ldots, T\}$	Deliberation round index
$s_i^t \in \mathbb{R}^{d_{\text{agent}}}$	Agent i's local state at round t
W_i^{proj}	Agent i's learned perspective-projection matrix (Eq. 1)
r_i	Agent i's learned role embedding (Eq. 1)
$\hat{R}$	Row-normalized stack of all role embeddings (Eq. 2)
$\mathscr{L}_{\text{orth}}$	Orthogonality regularizer on role embeddings (Eq. 2)
m_i^t	Message emitted by agent i at round t (Eq. 3)
ϕ	Shared feed-forward message-encoding network
e_i^t	Agent i's evidence vector at round t, derived from α_i^t
a_{ij}	Attention weight from agent i to message m_j^t (Eq. 4)
W_q, W_k	Query/key projection matrices for inter-agent attention
d	Attention key/query dimensionality
λ_D	Disagreement-attraction coefficient (Eq. 4)
D_{ij}	L1 distance between agents i, j's current beliefs
λ_S	Redundancy-suppression coefficient (Eq. 4)
S_{ij}	Cosine similarity between agents i, j's evidence vectors
$b_i^t \in (0, 1)$	Agent i's belief at round t (Eq. 5)
$u_i^t \in [0, 1]$	Agent i's self-uncertainty at round t (Eq. 5)
$\sigma(\cdot)$	Logistic sigmoid function
α_i^t	Agent i's accumulated Dirichlet evidence parameters
ρ_i	Exponential moving average of agent i's training-time accuracy
β_u, β_r	Consensus weighting coefficients for uncertainty / reliability (Eq. 6)
w_i	Agent i's consensus weight (Eq. 6)
p	Final consensus belief (imaging setting; Eq. 6)
p^*	Arbitrary reference distribution (Proposition 1)
p_{PoE}	Normalized log-linear (product-of-experts) pool
P	Number of virtual plan tokens (language setting; Section 3.6)
d_{llm}	Frozen decoder's input embedding dimensionality

B Full Algorithmic Specification

Algorithm 1 gives the complete forward pass of the CIDA module, combining Equations 1–6 into a single procedure, for the imaging (classification) setting. The language setting is identical through the end of the deliberation loop (line 23); it then branches to project the converged agent states into P plan tokens (Section 3.6) instead of computing a scalar consensus belief.

Three implementation notes on Algorithm 1 that are not visible in the main-text equations. First, InitBelief and AccumulateEvidence are small learned sub-networks whose exact architecture (number of layers, hidden width) we have not fixed as part of the core architectural claims of this paper and which we therefore do not specify further here; they are implementation details of the released code rather than claims of this paper. Second, the inner double loop over (i, j) pairs is $O(M^2)$ per round, which is negligible for M=4 but would need revisiting for a version of CIDA with a much larger agent count (see "Adaptive agent and round counts," Section 9). Third, ρ_i (agent i's reliability) is a parameter accumulated across the entire training run, not recomputed within a single forward

Algorithm 1 CIDA forward pass (imaging / classification setting)

Require: context vector h; trained parameters $\{W_i^{\text{proj}}, r_i, W_q, W_k, \phi, \dots\}$; rounds T; agents $M{=}4$
Ensure: consensus belief p

for $i = 1$ to M **do** ▷ Perspective projection, Eq. 1
  $s_i^0 \leftarrow W_i^{\text{proj}} h + r_i$
  $b_i^0, \alpha_i^0 \leftarrow \textsc{InitBelief}(s_i^0)$
end for
for $t = 0$ to $T-1$ **do**
  for $i = 1$ to M **do** ▷ Contrastive communication, Eqs. 3–4
    $e_i^t \leftarrow \textsc{EvidenceVec}(\alpha_i^t)$
    $m_i^t \leftarrow \phi([s_i^t; e_i^t])$
  end for
  for $i = 1$ to M **do**
    for $j = 1$ to M, $j \neq i$ **do**
      $D_{ij} \leftarrow \|b_i^t - b_j^t\|_1; \quad S_{ij} \leftarrow \cos(e_i^t, e_j^t)$
      $a_{ij} \leftarrow \exp\big(\langle W_q s_i^t, W_k m_j^t \rangle / \sqrt{d} + \lambda_D D_{ij} - \lambda_S S_{ij}\big)$
    end for
    $a_{i,\cdot} \leftarrow \text{softmax}(a_{i,\cdot})$ ▷ normalize over $j \neq i$
    $\tilde{m}_i^t \leftarrow \sum_{j \neq i} a_{ij} m_j^t$ ▷ aggregated incoming message
  end for
  for $i = 1$ to M **do** ▷ State update, Eq. 5
    $s_i^{t+1} \leftarrow \textsc{GatedFusion}(s_i^t, \tilde{m}_i^t, e_i^t)$
    $b_i^{t+1} \leftarrow \sigma(s_i^{t+1}); \quad u_i^{t+1} \leftarrow 4 b_i^{t+1}(1 - b_i^{t+1})$
    $\alpha_i^{t+1} \leftarrow \textsc{AccumulateEvidence}(\alpha_i^t, s_i^{t+1})$
  end for
end for
for $i = 1$ to M **do** ▷ Reliability-weighted consensus, Eq. 6
  $w_i \leftarrow \exp(-\beta_u u_i^T + \beta_r \rho_i)$
end for
$w \leftarrow w / \sum_i w_i$ ▷ normalize to sum to 1
$p \leftarrow \sigma\big(\sum_i w_i \log b_i^T\big)$
return p

pass; at inference time it is read as a fixed, frozen scalar per agent.

C Proof of Proposition 1

We restate the proposition for convenience. Let $b_1, \dots, b_M$ be probability distributions over a common outcome space, let $w_1, \dots, w_M \geq 0$ with $\sum_i w_i = 1$, and define the normalized log-linear pool

$$p_{\text{PoE}}(x) = \frac{\prod_i b_i(x)^{w_i}}{Z}, \qquad Z = \sum_x \prod_i b_i(x)^{w_i}. \tag{9}$$

For any reference distribution p^* over the same outcome space, we claim

$$\text{KL}(p^* \,\|\, p_{\text{PoE}}) \leq \sum_i w_i \,\text{KL}(p^* \,\|\, b_i). \tag{10}$$

Proof. By definition of KL divergence,

$$\mathrm{KL}(p^* \| p_{\mathrm{PoE}}) = \sum_x p^*(x) \log \frac{p^*(x)}{p_{\mathrm{PoE}}(x)} \tag{11}$$

$$= \sum_x p^*(x) \log p^*(x) - \sum_x p^*(x) \log \left(\frac{\prod_i b_i(x)^{w_i}}{Z} \right) \tag{12}$$

$$= \sum_x p^*(x) \log p^*(x) - \sum_i w_i \sum_x p^*(x) \log b_i(x) + \log Z. \tag{13}$$

Now observe that $\sum_i w_i \mathrm{KL}(p^* \| b_i) = \sum_i w_i \sum_x p^*(x) \log \frac{p^*(x)}{b_i(x)} = \sum_x p^*(x) \log p^*(x) - \sum_i w_i \sum_x p^*(x) \log b_i(x)$, using $\sum_i w_i = 1$. Subtracting the previous display from this expression,

$$\sum_i w_i \mathrm{KL}(p^* \| b_i) - \mathrm{KL}(p^* \| p_{\mathrm{PoE}}) = -\log Z. \tag{14}$$

It remains to show $-\log Z \geq 0$, i.e., $Z \leq 1$. This is exactly where Hölder's inequality is used: for a probability distribution x (here, uniform counting measure over the discrete outcome space, or more generally the base measure with respect to which b_i and p^* are densities) and non-negative weights w_i summing to one, the generalized Hölder inequality gives

$$\sum_x \prod_i b_i(x)^{w_i} \leq \prod_i \left(\sum_x b_i(x) \right)^{w_i} = \prod_i 1^{w_i} = 1, \tag{15}$$

since each b_i is itself a normalized probability distribution, $\sum_x b_i(x) = 1$. Hence $Z \leq 1$, so $-\log Z \geq 0$, so $\sum_i w_i \mathrm{KL}(p^* \| b_i) - \mathrm{KL}(p^* \| p_{\mathrm{PoE}}) \geq 0$, which is the claimed inequality. □

Remark 2. *The proof uses only that each b_i is a valid normalized probability distribution and that the weights w_i are non-negative and sum to one; it does not use anything about* how *the w_i were chosen (in particular, it does not use the specific uncertainty/reliability weighting of Equation 6), and it does not use any independence assumption between the b_i. This is precisely why the remark in Section 3.5 states that the proposition is a general property of log-linear pooling, not a property specific to CIDA's particular weighting scheme.*

D Extended Qualitative Method Comparison

Table 9 extends Table 1 with three additional axes: whether the method's per-inference cost scales with the number of sub-agents or search branches involved, whether the method has been evaluated (by its original authors) at scales comparable to production language models, and a one-line note on the primary trade-off each method makes, as characterized by its own paper.

We want to draw particular attention to the "evaluated at production scale" column for CIDA: unlike every other row in this table, whose original papers evaluated at scales ranging from moderate to large production language models, every CIDA result in this paper was obtained under the single-GPU, Colab-based compute budget described in Section 4.4. This is the most important single caveat governing how Table 9 (and Table 1) should be read: CIDA's position in this design-space map reflects where the *architecture* sits conceptually, not a claim that it has been validated at a scale comparable to the other rows.

E Hyperparameters and Reproducibility

Table 10 lists the hyperparameters and settings we can currently confirm from our logs for the primary reported configurations, together with an explicit "Unconfirmed" marker for values that are referenced in the main text (Section 3) but that we cannot currently pin to a single exact number with confidence, pending the log reconciliation discussed in Appendix F. We include the unconfirmed rows rather than omit them, on the view that an explicit "we do not currently know this precisely" is more useful to a reproducing researcher than silence.

Table 9: Extended qualitative comparison. Compiled from the cited papers' own descriptions; not an empirical re-evaluation by us (see also Table 1).

Method	Scales with agents/branches?	Evaluated at production scale?	Primary trade-off (per original paper)
CoT [1]	No (single trace)	Yes	Accuracy for token cost and latency
Tree of Thoughts [2]	Yes (branch count)	Partially	Accuracy for a large multiple of token cost
RL reasoning [3]	No (single trace, learned length)	Yes	Accuracy for self-determined, sometimes very long, trace length
Debate [4]	Yes ($M \times R$)	Partially	Factuality for $M \times R$ generation cost
Coconut [5]	No	Partially	Token efficiency for reduced interpretability
Quiet-STaR [6]	No (parallel internal rationales)	Partially	Reasoning quality for added training complexity
Pause tokens [7]	No	Partially	Extra compute headroom for no added interpretability
Prompt/prefix tuning [8, 9]	No	Yes	Parameter efficiency for reduced expressivity vs. full fine-tuning
CIDA (this work)	Yes (M agents, fixed here at 4)	**No (feasibility study only)**	Token/latency efficiency and structured multi-perspective computation for reduced interpretability and (currently) unconfirmed attribution vs. equal-capacity controls

Table 10: Hyperparameters and settings. "Confirmed" entries are cross-checked against raw logs for this draft; "Unconfirmed" entries are referenced in the main text but not yet independently verified against logs.

Setting	Value	Applies to	Status
Agents M	4	All configurations	Confirmed
Deliberation rounds T	4 (as plotted, Fig. 3)	Illustrative only	Unconfirmed (design target)
Plan tokens P	4–8	Language settings	Confirmed (range only)
CheXpert subset size	50,000 images	Imaging	Confirmed
CheXpert seed	42	Imaging	Confirmed
CheXpert epochs	15	Imaging	Confirmed
CheXpert batch size	32	Imaging	Confirmed
Optimizer	AdamW, cosine schedule	Imaging & language	Confirmed
GPU	1× A100 (40GB) or 1× T4	All configurations	Confirmed
Uncertain-label mapping	U-Ones (map to positive)	Imaging	Confirmed
ECE bin count	10–15	Imaging	Confirmed (range only)
λ_D, λ_S (Eq. 4)	—	All configurations	Unconfirmed (not tuned systematically)
β_u, β_r (Eq. 6)	—	All configurations	Unconfirmed
Orthogonality loss weight (Eq. 2)	—	All configurations	Unconfirmed
v1 adapter parameters	67M	Qwen1.5-7B-Chat	Unconfirmed†
v1.5 adapter parameters	160M	Qwen-7B	Unconfirmed†
v2 adapter parameters	17M	Qwen1.5-1.8B	Unconfirmed†
v3 adapter parameters	$<$1M	Qwen2.5-1.5B	Confirmed (upper bound only)

We plan, as part of the near-term action items in Section 9, to release a fully reconciled hyperparameter and configuration file alongside the code repository, so that every "Unconfirmed" row in Table 10 can eventually be replaced with an exact, log-verified value or an explicit statement that the corresponding run was not systematically tuned.

F Author Checklist

This checklist was maintained during preparation of this paper and is reproduced here, essentially verbatim, rather than silently resolved, in keeping with the emphasis throughout this paper on making the boundary between measured and unmeasured claims explicit. Items are listed in the order they were raised; several are referenced directly from the main text.

1. **Headline AUC.** An earlier draft used 0.939 in the abstract and headline table, while figure titles in the source files instead read 0.9385, and the per-class values shown in the ROC legend (Figure 4) average to approximately 0.912. These three numbers should, in principle, be the same quantity

computed three different ways from the same underlying predictions. We use $0.939^{\dagger}$ throughout this draft as the best current estimate, mark it with a dagger everywhere it appears, and treat full reconciliation against the raw prediction logs as an open item.

2. **Ablation baseline.** The ablation figure (Figure 6) shows a baseline AUC of 0.812, while Table 3 uses a reported literature baseline of 0.847 for plain DenseNet-121. We confirm here explicitly that these are two different rows: 0.812 is our own frozen-DenseNet run under our specific 50k-subset protocol (used as the "no CIDA at all" reference point for the ablation), while 0.847 is the externally reported figure used as a point of comparison in Table 3. An earlier draft's ablation figure caption stated a "50k validation set," while an earlier draft of what is now Table 4 stated "10k images, 5 epochs"; we confirm the setting used for every number in Table 4 and Figure 6 is the same 50k-subset, 15-epoch protocol described in Section 5, and the earlier "10k, 5 epochs" description was an error in that earlier draft, not a separate run.
3. **Patient-level split.** We have not yet confirmed, from our own data-splitting code, that the CheXpert train/validation split used for this paper's results is a patient-level split (i.e., that no single patient's images appear in both the training and validation partitions). This matters because a study-level or image-level split that inadvertently leaks the same patient across train and validation could inflate reported validation performance. We flag this as an unresolved, and important, open item rather than assume the standard convention was followed correctly.
4. **Reliability diagram.** An earlier draft's Figure 3 was captioned as a set of "reliability diagrams," but the actual image was a per-class accuracy/F1 bar chart (now Figure 5 in this draft, correctly captioned). No true reliability diagram (predicted-confidence-vs-empirical-accuracy plot, binned as in Equation 8) currently exists for this paper. Generating one from the logged predictions, with 10–15 confidence bins, is listed as a near-term item in Section 9 for anyone wishing to visually inspect CIDA's calibration beyond the scalar ECE number.
5. **Adapter sizes.** The 67M / 160M / 17M / $<$1M adapter parameter counts in Table 2 are copied from earlier internal source tables; we have not independently re-derived each figure from the corresponding model's saved checkpoint size, and Appendix E marks all but the v3 figure as unconfirmed pending that check.
6. **Removed figures.** An earlier draft's Figure 6 (purporting to show per-agent belief trajectories from real inputs) and Figure 7 (purporting to show per-agent attention maps) exhibited clear signs of having been produced by an unconstrained generative image process rather than logged from the model — a visible generator watermark, and misspelled role labels ("Skteptic," "Integrax") that do not match this paper's own terminology, along with duplicated panels. These must not be presented as model output under any circumstances. The former Figure 7 has been removed from this draft entirely; the former Figure 6 has been replaced by the explicitly labeled conceptual schematic now shown as Figure 3, captioned throughout this draft as *not* measured output. Producing real versions of either figure requires the per-agent logging infrastructure listed as a near-term item in Section 9, which does not yet exist.
7. **Answer extraction.** The answer-parsing code used for the language-reasoning baselines has not been audited, and the very low base (no-CoT) scores on several BIG-Bench Hard subtasks (Section 6.3) may be partly attributable to parser failures rather than the base model's inability to solve the underlying problems. Flagged as a near-term audit item in Section 9.
8. **Efficiency numbers.** The efficiency numbers reported in Table 6 are drawn from our own source measurement table. A small number of additional efficiency claims that appeared in an earlier internal draft — specifically a $4.77\times$ speedup figure, a -14% figure, a -72% figure, and a per-token cost claim — were dropped from this version because they conflicted with the source table we could verify, and we could not reconcile the discrepancy before finalizing this draft. Only the four rows shown in Table 6, which we can trace directly to a verified measurement, are reported here.

This checklist is, by design, incomplete in the sense that new items may surface as the near-term action items in Section 9 are carried out; we intend to maintain and publish an updated version of this checklist alongside any future revision of this paper or its accompanying code release.